THE DISCRETE FOURIER TRANSFORM OVER PRIME FIELDS BELOW $N \log N$ ON MULTITAPE MACHINES

DAN BROWN*

Abstract. A fixed finite multi-stack Turing machine, in the model `Turing.FinTM2` of Lean's Mathlib, computes the discrete Fourier transform of length n over a prime field $\mathbb{F}_p$, given p, a primitive n-th root of unity and the n entries in binary of fixed width, in worst-case time $O\big(N(\log N)^{1-\kappa}\big)$, where N is the input length and $\kappa > 0$ is the saving of the formally verified integer multiplier below $n \log n$. The reduction is the standard reduction of the transform to one integer product: Bluestein's chirp, written with binomial coefficients so that no square root of the root of unity is needed, Kronecker packing, and Barrett reduction with a reciprocal computed by Newton's iteration. The contribution is the explicit sub-$N \log N$ bound on multitape machines, inherited from the multiplier, and a statement of it in Lean that uses Mathlib definitions only. The Lean proof is in progress; we list what it proves.

1. Introduction

For a prime p, an integer $n \geq 1$, a primitive n-th root of unity $\omega \in \mathbb{F}_p$ and a vector $a \in \mathbb{F}_p^n$, the discrete Fourier transform is

$$\mathrm{DFT}_\omega(a)_k = \sum_{j=0}^{n-1} a_j \, \omega^{jk} \qquad (0 \leq k < n).$$

Over a field whose transform lengths are smooth, the fast Fourier transform computes it with $O(n \log n)$ field operations. The bit complexity on a multitape Turing machine is a different question: the field elements have $w = \mathrm{size}(p)$ bits, n need not be smooth, and the machine must move its data along tapes. The classical route reduces the transform to a single integer multiplication of $O(N)$ bits, where $N \asymp nw$ is the input length, so the transform costs the time of that product plus $O(N)$ for the surrounding passes and $O(n)$ multiplications of w-bit numbers.

The time of integer multiplication on multitape machines has been bounded by $O(n \log n)$ by Harvey and van der Hoeven [3], and below $n \log n$ by the OpenAI manuscript [7]: time $O\big(n(\log n)^{1-\kappa}\big)$ for a positive κ. A complete formal proof of a bound of this form, for one fixed machine in Mathlib's multi-stack model, is the Lean development `IntegerMultBounds` [4]; its theorem `MathlibTM2.mulInTime` gives the saving $\kappa = 88279 \cdot 2^{-46} \approx 2^{-29.57}$. The reduction then gives the same saving for the transform.

Theorem. *Let $\kappa > 0$ be such that some fixed finite multi-stack machine multiplies natural numbers in time $O\big(n(\log n)^{1-\kappa}\big)$, with binary inputs and outputs. Then a fixed finite multi-stack machine computes $\mathrm{DFT}_\omega(a)$ for every prime p, every primitive n-th root of unity $\omega \in \mathbb{F}_p$ and every $a \in \mathbb{F}_p^n$, with the input written as p, ω, $a_0, \ldots, a_{n-1}$ in exactly $\mathrm{size}(p)$ binary digits each, in time*

$$O\big(N(\log N)^{1-\kappa}\big),$$

where $N = (n+2)(\mathrm{size}(p)+1)$ is the input length. The implicit constant depends neither on p nor on n. In particular this holds with $\kappa = 88279 \cdot 2^{-46}$.

The bound is uniform in the relation between n and p: it covers $n = 2$ with p of millions of bits as well as $n = p - 1$. The first regime requires that every one-time setup on p costs $O(M(w))$, the time of one

Date: October 17, 2026.

*Author of *The Da Vinci Code*.

product of w-bit numbers, without a logarithmic number of such products; Section 4 explains where this forces the choice of the modular reduction.

Section 2 gives the statement in Lean and explains why the inputs have fixed width. Section 3 gives the reduction, Section 4 the cost accounting, and Section 5 the state of the formalization.

2. The statement

The Lean statement is `DFTStatement` in `MathlibTM2/DFT/Statement.lean`. Apart from the few definitions shown, everything in it is Mathlib vocabulary: the alphabet Γ', the encoding `encodeNat` of $\mathbb{N}$ in binary, `ZMod`, `IsPrimitiveRoot`, `TM2ComputableInTime` and the asymptotic relation `=O[atTop]`. Numbers are written least significant bit first, each followed by a comma.

```
def encodeFixed (w x : ℕ) : List Γ' :=
  List.ofFn fun i : Fin w => Γ'.bit (x.testBit i)

def encodeFixedList (w : ℕ) (l : List ℕ) : List Γ' :=
  l.flatMap fun x => encodeFixed w x ++ [Γ'.comma]

def encodeNatList (l : List ℕ) : List Γ' :=
  l.flatMap fun x => (encodeNat x).map inclusionBoolΓ' ++ [Γ'.comma]

def dft {p n : ℕ} (ω : ZMod p) (a : Fin n → ZMod p) (k : Fin n) : ZMod p :=
  ∑ j, a j * ω ^ ((j : ℕ) * k)

structure DFTInstance where
  p : ℕ
  [prime : Fact p.Prime]
  n : ℕ
  ω : ZMod p
  primitive : IsPrimitiveRoot ω n
  a : Fin n → ZMod p

def DFTInstance.encode (x : DFTInstance) : List Γ' :=
  encodeFixedList (Nat.size x.p) (x.p :: x.ω.val :: List.ofFn fun j => (x.a j).val)

def DFTInstance.transform (x : DFTInstance) : List ℕ :=
  List.ofFn fun k => (dft x.ω x.a k).val

def DFTInTime (κ : ℝ) : Prop :=
  ∃ M : TM2ComputableInTime DFTInstance.encode encodeNatList DFTInstance.transform,
    (fun N : ℕ => (M.time N : ℝ)) =O[atTop]
      fun N : ℕ => (N : ℝ) * Real.log N ^ (1 - κ)

def DFTStatement : Prop := ∃ κ : ℝ, 0 < κ ∧ DFTInTime κ
```

`TM2ComputableInTime` is Mathlib's notion: a machine of type `Turing.FinTM2`, a time function of the input length, and a proof that on every input the machine halts with the encoded output within that time. The time function therefore bounds the worst case over all instances of a given length. The output is the list of canonical representatives $\mathrm{DFT}_\omega(a)_k \in \{0, \dots, p-1\}$ in shortest binary form; its length is at most $n(\mathrm{size}(p)+1) \le N$.

Remark 2.1 (Fixed-width inputs)**.** *With shortest binary forms on input as well, no bound $O\big(N(\log N)^{1-\kappa}\big)$ is possible. Take $n = p-1$, a generator ω of $\mathbb{F}_p^\times$, and $a = (0,1,0,\dots,0)$. In shortest form each zero entry is a single comma, so the input has about $p + 2\log_2 p$ symbols. The transform is $\mathrm{DFT}_\omega(a)_k = \omega^k$, which runs over all of $\mathbb{F}_p^\times$, so the output has $\sum_{x=1}^{p-1}(\mathrm{size}(x)+1) \asymp p \log p$ symbols. A machine writes at most one output symbol per step, so its time on these inputs is at least of order $N \log N$. With every entry written in exactly $\mathrm{size}(p)$ digits, the output is never longer than the input, and this lower bound disappears.*

3. The reduction

Fix an instance with $n \geq 2$ and write $w = \mathrm{size}(p)$, so $2^{w-1} \leq p < 2^w$. Since ω has order n in $\mathbb{F}_p^\times$, $n \mid p-1$; hence $n < p$, $\mathrm{size}(n) \leq w$, and p is odd. The cases $n \leq 1$ are immediate: for $n = 0$ the output is empty, and for $n = 1$ we have $\omega = 1$ and $\mathrm{DFT}_\omega(a) = a$.

3.1. **The chirp without a square root.** Bluestein [2] turns the transform into a correlation through the identity $jk = \frac{1}{2}\big((j+k)^2 - j^2 - k^2\big)$, which uses a square root of ω. The binomial form avoids it:

$$jk = \binom{j+k}{2} - \binom{j}{2} - \binom{k}{2},$$

an identity of integers. Put $\omega' = \omega^{n-1}$, so $\omega'\omega = 1$, and $c_m = \omega^{\binom{m}{2}}$, $c'_m = \omega'^{\binom{m}{2}}$. Then $\omega^{jk} = c'_j\, c'_k\, c_{j+k}$, and

$$\mathrm{DFT}_\omega(a)_k = c'_k \sum_{j=0}^{n-1} u_j\, c_{j+k}, \qquad u_j = a_j\, c'_j.$$

The sum is a correlation of the vector u of length n with the chirp c of length $2n-1$.

3.2. **Kronecker packing.** Let $\bar{u}_j, \bar{c}_m \in \{0,\dots,p-1\}$ be the representatives and $B = 2^s$ with $s = 2w + \mathrm{size}(n)$, so that $n(p-1)^2 < B$. Pack the reversed vector and the chirp as integers,

$$U = \sum_{i=0}^{n-1} \bar{u}_{n-1-i}\, B^i, \qquad C = \sum_{m=0}^{2n-2} \bar{c}_m\, B^m.$$

The coefficient of B^{n-1+k} in the polynomial product is $T_k = \sum_{j=0}^{n-1} \bar{u}_j\, \bar{c}_{j+k}$, and every coefficient of the product is a sum of at most n terms below $(p-1)^2$, hence below B. So no carries occur, and T_k is the digit of index $n-1+k$ of the integer UC in base B:

$$T_k = \big\lfloor UC / B^{n-1+k} \big\rfloor \bmod B.$$

The two factors have ns and $(2n-1)s$ bits, so the single product has input length at most $3ns + O(1) = O(N)$. Then $\mathrm{DFT}_\omega(a)_k = c'_k\, T_k$ in $\mathbb{F}_p$.

3.3. **Reduction modulo p.** The remaining arithmetic is $O(n)$ products of residues and n reductions of $T_k < np^2$. Both are Barrett reductions [1]. With $\mu = \lfloor 2^{s'}/p \rfloor$ and $x < 2^{s'}$, the quotient $q = \lfloor x\mu / 2^{s'} \rfloor$ satisfies

$$qp \leq x < qp + 2p,$$

so $x \bmod p$ is $x - qp$ after at most two subtractions of p. One reduction costs two products of $O(s')$-bit numbers. We use $s' = 2w$ for products of residues and $s' = s$ for the sums T_k.

The constant μ is computed by Newton's iteration for the reciprocal with precision doubling [5, §4.3.3]. For a divisor d of $2k$ bits and a the reciprocal of its top k bits at precision k, the step

$$y = 2^{k+1}a - \big(\lfloor da^2 / 4^k \rfloor + 1\big)$$

satisfies $yd < 16^k < (y+9)d$, so the reciprocal $\lfloor 16^k/d \rfloor$ at precision $2k$ is y plus at most eight corrections. The levels have precisions $1, 2, 4, \dots$ up to $O(w)$, each with three products, so the total is a geometric sum, $O(M(w))$.

Remark 3.1 (Montgomery reduction)**.** *Montgomery's reduction* [6] *replaces division by p with division by $R = 2^r$, $R > np$, given the constant $-p^{-1} \bmod R$. That constant is computed by Hensel lifting $y \mapsto y(2 + py)$ in $O(M(w))$ time, again a geometric sum. Entering Montgomery form, however, needs $R^2 \bmod p$, a reduction of a $2r$-bit number modulo p. Computed by $\log_2 r$ Montgomery squarings, it costs $O(M(w) \log w)$, which exceeds the budget of Section 4 by a factor of order $\log w$ when n is bounded. Computing it in $O(M(w))$ requires a division by p, and with the reciprocal μ in hand Barrett reduction performs every reduction of the pipeline directly.*

4. Cost

Let $M(L)$ be the time of the multiplier on inputs of total length L, replaced by a monotone majorant with $M(L) \le C_0\, L(\log(L+2))^{1-\kappa} + C_0$; then $M(L)/L$ is bounded by a nondecreasing function, and $M(cL) = O(M(L))$ for fixed c. Every product of $O(w)$-bit numbers costs $O(M(w))$ including the transfer of its operands and its result, and every pass over the data costs $O(N)$. With $N = (n+2)(w+1)$ and $n \ge 2$:

(1) reading the input and printing the output: $O(N)$;
(2) the reciprocals $\lfloor 4^w/p \rfloor$ and $\lfloor 2^s/p \rfloor$ by Newton's iteration: $O(M(w))$;
(3) $\omega' = \omega^{n-1}$ by repeated squaring: $O(\log n \cdot M(w))$;
(4) the chirps c_m $(m < 2n-1)$ and c'_m $(m < n)$, from $c_{m+1} = c_m\, \omega^m$ and $\omega^{m+1} = \omega^m \omega$: $O(n\, M(w))$;
(5) the chirped input $u_j = a_j c'_j$, packed in reverse: $O(n\, M(w) + N)$;
(6) the product UC: $M(9N)$, since $3ns \le 9nw \le 9N$;
(7) the digits T_k, their reductions and the products $c'_k T_k$: $O\big(n\, M(w) + N\big)$, since $s \le 3w$.

Since $nw \le N$ and $M(w)/w$ is bounded by $O\big((\log N)^{1-\kappa}\big)$, each term $n\, M(w)$, $\log n \cdot M(w)$ and $M(w)$ is $O\big(N(\log N)^{1-\kappa}\big)$, and so is $M(9N)$. The total is $O\big(N(\log N)^{1-\kappa}\big)$, uniformly in n and p.

Remark 4.1 (Tiny n, huge p)**.** *For $n = 2$ the input length is $N = 4(w+1)$ and the budget is $O\big(w(\log w)^{1-\kappa}\big)$, the time of a constant number of products of w-bit numbers. Every step of the list fits: the Newton reciprocal is a geometric sum of products, and the other steps use $O(n)$ products. A setup with $\Theta(\log w)$ products of w-bit numbers does not fit, which is the reason for the choice in Remark 3.1. For n close to p the product UC dominates.*

5. Formalization

The statement of Section 2 is in `MathlibTM2/DFT/Statement.lean` of the repository [4], which imports Mathlib only; its examples check `encodeFixedList`, `encodeNatList` and one transform over $\mathbb{F}_5$ by `decide`. The proof of `DFTStatement` is in progress. The development takes the multiplier as a parameter, a structure `HostMultiplier` κ holding a machine table, its time function and the bound $O\big(n(\log n)^{1-\kappa}\big)$, so nothing in it depends on the value of κ. The following parts are proved, with no `sorry`:

- `DFT/Algorithm.lean`: the arithmetic of Section 3 on natural numbers. `chirp_identity` is the binomial form of Bluestein's identity with $\omega'\omega = 1$; `pack_mul`, `pack_div_pow_mod`, `conv_le` and `conv_reflect` are Kronecker packing and digit read-back; `correlation_cast` allows arbitrary representatives; `fastDFT_transform` states that the whole computation, with reductions modulo p, equals `DFTInstance.transform` when $n(p-1)^2 < B$.
- `DFT/Reduction.lean` and `DFT/Newton.lean`: Montgomery's REDC (`redc_lt`, `redc_cast`), the Hensel constant (`montInv_dvd`), Barrett reduction (`barrett_mod`, `barrett2_le`, `barrett2_lt`) and the Newton step (`newton_step`).
- `DFT/Keys.lean`: a machine with two Γ' stacks around the host register file of [4], the transfer of host programs (`exec_hostProg`), and the packaging into `TM2ComputableInTime` and `DFTInTime` (`computableInTime`, `dftInTime_of_computes`).

- `DFT/IO.lean`, `DFT/Fields.lean`, `DFT/Parse.lean`: reading the fixed-width input and printing the output (`parse_exec`, `print_exec`).
- `DFT/MulIO.lean`, `DFT/MulCall.lean`: the multiplier as a subroutine on internal registers (`call_exec`, `mulCall_exec`).
- `DFT/Trunc.lean`, `DFT/RedcTail.lean`, `DFT/Redc.lean`, `DFT/Montgomery.lean`: REDC and Montgomery multiplication on the host (`redc_exec`, `montMul_exec`).
- `DFT/Hensel.lean`, `DFT/HenselHost.lean`, `DFT/HenselLoop.lean`: the Hensel iteration with its step bound (`henselRound_exec`, `hensel_exec`).
- `DFT/NewtonHost.lean`, `NewtonSetup.lean`, `NewtonBlocks.lean` and `NewtonGuess.lean`: the host blocks of one Newton level (`newtonPrep_exec`, `newtonLoad_exec`, `newtonSq_exec`, `newtonGuessProg_exec`, `newtonFinish_exec`) and the correction loop (`corrLoop_exec`).

Still to be written are the Newton iteration over all levels and Barrett reduction on the host, the pipeline of Sections 3 and 4 as one host procedure with its step bound, the top level that turns it into `HostMultiplier` $\kappa \to$ `DFTInTime` κ, and the final theorem `dft_in_time : DFTStatement`, obtained from the multiplier of `MathlibTM2.mulInTime`, with its axiom audit.

References

[1] P. Barrett, *Implementing the Rivest Shamir and Adleman public key encryption algorithm on a standard digital signal processor*, Advances in Cryptology – CRYPTO '86, Lecture Notes in Comput. Sci. **263**, Springer, 1987, 311–323.

[2] L. I. Bluestein, *A linear filtering approach to the computation of discrete Fourier transform*, IEEE Trans. Audio Electroacoust. **18** (1970), 451–455.

[3] D. Harvey, J. van der Hoeven, *Integer multiplication in time $O(n \log n)$*, Ann. of Math. **193** (2021), 563–617.

[4] `IntegerMultBounds`: integer multiplication below $n \log n$ in Lean on Mathlib's finite Turing machines, https://github.com/platypii/integer-mult-bounds-lean.

[5] D. E. Knuth, *The Art of Computer Programming*, Vol. 2: *Seminumerical Algorithms*, 3rd ed., Addison-Wesley, 1997.

[6] P. L. Montgomery, *Modular multiplication without trial division*, Math. Comp. **44** (1985), 519–521.

[7] OpenAI, *Integer multiplication below $n \log n$*, manuscript, September 23, 2026, https://github.com/openai/math.

Email address: jdb1729@gmail.com